LOVELY PROFESSIONAL UNIVERSITY

PROJECT PROGRESS REPORT

GYM CLASS CAPACITY AND WAITLIST MANAGER

CA1 – DESIGN & CREAT

Submitted By
Vinit Kumar
Registration Number: 12604409
Programme / Section: MCA / D1P2635
Roll Number: 29

Course
CAP455: Object Oriented Programming Using C++

TABLE OF CONTENTS

Table of Contents 2
1. Abstract 3
2. Introduction & Problem Statement 3
  2.1 Introduction 3
  2.2 Problem Statement 3
3. Objectives of the System 3
4. Entities and Class Design 4
  4.1 Main Entity – GymClass 4
  4.2 Attributes of GymClass 4
  4.3 Main Class 4
  4.4 Unique Key 4
5. Software & Hardware Requirements 5
  5.1 Software Requirements 5
  5.2 Hardware Requirements 5
6. Object-Oriented Concepts Implemented 5
7. System Design & Architecture 6
  7.1 UML Class Diagram 6
  7.2 System Flowchart 6
8. Module Description 7
  8.1 Class Management Module 7
  8.2 Add Record Module 7
  8.3 Display Records Module 7
  8.4 Search Module 7
  8.5 Available Seats Module 7
  8.6 Record Counter Module 7
9. Data Dictionary and Function Description 8
10. Sample Data & Expected Output 9
11. Source Code 10–13
12. Test Cases and Limitations 14
  12.1 Test Cases 14
  12.2 Current Limitations 14
  12.3 Future Scope 14
13. CA1 Progress Summary & Conclusion 15
  13.1 Progress Summary 15
  13.2 Conclusion 15

1. ABSTRACT

The Gym Class Capacity and Waitlist Manager is a console-based application developed using C++. Its purpose is to organize information about gym classes, trainers, class capacity, enrolled members and available seats.

Gyms commonly provide fitness classes such as Yoga, Zumba, Cardio and Weight Training. Each class has a limited capacity. Managing these details manually can lead to duplicate records, incorrect seat counts and difficulty locating class information.

The CA1 version establishes the foundation of the application. It allows a user to add a gym class, display all stored class records and search for a class by its unique ID. It demonstrates classes, objects, static members, inline functions, friend functions, arrays of objects, pointers, strings and the this pointer.

Records are stored temporarily in a fixed-size array during program execution. Permanent file storage and full member waitlist operations are planned for later project phases.

2. INTRODUCTION & PROBLEM STATEMENT

2.1 Introduction

A gym may conduct several fitness sessions every day. Each session has a name, trainer, fee and maximum number of participants. Staff members need a reliable way to check how many members are enrolled and how many seats remain.

This project provides a simple menu-driven C++ application to maintain class information in a structured form.

2.2 Problem Statement

Manual records can become difficult to maintain as the number of gym classes increases. Duplicate IDs may be entered, available seats may be calculated incorrectly, and finding a particular class may take time. A small computerized application is proposed to organize class records and provide basic search and capacity information.

3. OBJECTIVES OF THE SYSTEM

- Maintain records of different gym classes.
- Assign a unique ID to every class and prevent duplicate IDs.
- Store class name, trainer, capacity, enrolled count and fee.
- Calculate available seats and show Open or Full status.
- Display all stored records and search by class ID.
- Demonstrate the C++ concepts required for CA1.
- Create a foundation for member registration and waitlist features in later phases.

4. ENTITIES AND CLASS DESIGN

4.1 Main Entity – GymClass

The main entity is GymClass. Each object represents one fitness class offered by the gym, for example Yoga, Zumba or Cardio.

4.2 Attributes of GymClass

Attribute	Data Type	Description
id	int	Unique identification number
name	string	Name of gym class
trainer	string	Assigned trainer
capacity	int	Maximum number of members
enrolled	int	Currently enrolled members
fee	float	Class fee
status	char	O = Open; F = Full

4.3 Main Class

The main class is named GymClass. Its data members are private to protect class information. Public member functions assign values, retrieve the ID, calculate available seats and access the record counter.

4.4 Unique Key

The id attribute acts as the unique key. No two gym class records should use the same ID. Before adding a new record, the program compares the entered ID with existing IDs. If a match is found, the duplicate record is rejected.

Class ID	Class Name	Trainer
101	Yoga	Rahul
102	Zumba	Priya
103	Cardio	Aman
104	Weight Training	Rohit

5. SOFTWARE & HARDWARE REQUIREMENTS

5.1 Software Requirements

- Programming language: C++.
- Compiler: GCC/G++ or Microsoft Visual C++.
- Language standard: C++11 or higher.
- Operating system: Windows 10/11 or compatible Linux.
- Development environment: Visual Studio Code, Code::Blocks or Visual Studio.

5.2 Hardware Requirements

- Processor: Intel Core i3 or equivalent.
- RAM: 2 GB minimum; 4 GB recommended.
- Storage: At least 50 MB free space.
- Keyboard and monitor.

6. OBJECT-ORIENTED CONCEPTS IMPLEMENTED

Concept	Use in this Project
Class and object	GymClass defines the structure; each record is an object.
Static data member	cnt stores the number of records added during execution.
Static functions	getCount() returns cnt; increaseCount() increments it.
Inline function	availableSeats() calculates capacity minus enrolled.
Friend function	show() accesses private members to display a record.
Array of objects	arr[50] stores up to 50 GymClass objects.
Pointer to object	ptr traverses the array during display.
this pointer	setData() uses this-> for current object's members.
String	name and trainer store text.
Call by reference	addRecord() receives n as int &n.

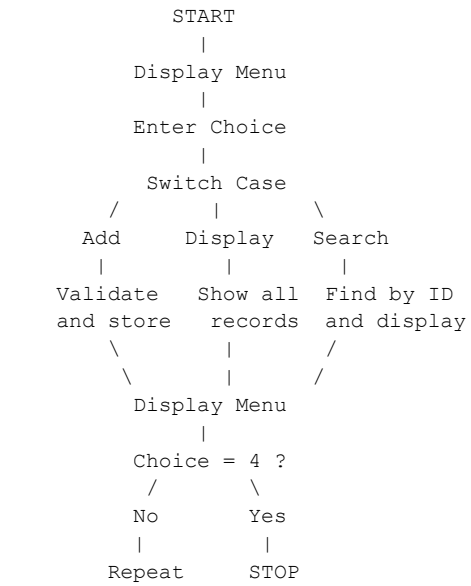
7. SYSTEM DESIGN & ARCHITECTURE

The CA1 application uses a menu-driven design. The main function displays the menu repeatedly using a do-while loop. A switch-case statement calls the function corresponding to the user's selection.

7.1 UML Class Diagram



7.2 System Flowchart



8. MODULE DESCRIPTION

8.1 Class Management Module

This module is based on GymClass. It stores the ID, class name, trainer, capacity, enrolled members, fee and status.

8.2 Add Record Module – addRecord()

This function accepts details of a new gym class. It checks the array limit, validates the positive class ID, checks duplicate IDs, accepts the remaining information and stores the record. The record count is then increased.

8.3 Display Records Module – displayAll()

This function displays every stored class record. It checks whether the array is empty and uses a pointer to object to move through the array.

8.4 Search Module – searchRecord()

This function accepts a class ID and compares it with each stored record. If a matching ID is found, the record is displayed; otherwise, a not-found message is shown.

8.5 Available Seats Module – availableSeats()

This inline function returns maximum capacity minus enrolled members. If capacity is 20 and enrolled members are 15, five seats are available.

8.6 Record Counter Module – cnt

The static data member cnt tracks records added during the current program run. increaseCount() increments it, and getCount() returns its value.

9. DATA DICTIONARY AND FUNCTION DESCRIPTION

9.1 Data Dictionary

Variable	Type	Purpose
id	int	Unique class ID
name	string	Class name
trainer	string	Trainer name
capacity	int	Maximum capacity
enrolled	int	Enrolled member count
fee	float	Class fee
status	char	Open or Full status
cnt	static int	Counts records added
arr	GymClass[50]	Fixed-size object array
ptr	GymClass*	Pointer to array objects
n	int	Number of stored records
choice	int	Menu selection
flag	int	Duplicate ID indicator
found	int	Search result indicator

9.2 Function Description

Function	Purpose
setData()	Assigns values to a GymClass object.
getId()	Returns the class ID.
availableSeats()	Calculates remaining seats.
getCount()	Returns total record count.
increaseCount()	Increases static record count.
show()	Displays a record using friend access.
addRecord()	Adds a new class record.
displayAll()	Displays all stored records.
searchRecord()	Searches by unique ID.
main()	Controls menu and program flow.

10. SAMPLE DATA & EXPECTED OUTPUT

10.1 Sample Gym Class Records

ID	Class	Trainer	Capacity	Enrolled	Fee
101	Yoga	Rahul	20	15	1000
102	Zumba	Priya	20	20	1200
103	Cardio	Aman	15	10	800
104	Weight Training	Rohit	25	18	1500

10.2 Sample Output – Add Class

```
===== GYM CLASS MANAGER =====
1. Add Gym Class
2. Display All Classes
3. Search Class by ID
4. Exit

Enter your choice: 1
Enter Class ID: 101
Enter Class Name: Yoga
Enter Trainer Name: Rahul
Enter Maximum Capacity: 20
Enter Enrolled Members: 15
Enter Class Fee: 1000

Record added successfully!
```

10.3 Sample Output – Display Record

```
ID: 101
Class Name: Yoga
Trainer: Rahul
Capacity: 20
Enrolled: 15
Available Seats: 5
Fee: 1000
Status: O
```

10.4 Sample Output – Search

When the user enters an existing ID, the program displays “Record Found!” and prints the corresponding class details. If the ID is not present, it displays “Record not found.”

11. SOURCE CODE – PART 1 OF 3

CA1 implementation using iostream, string and a fixed-size array. Records are maintained in memory during execution.

```
#include <iostream>
#include <string>
using namespace std;

class GymClass
{
private:
    int id;
    string name;
    string trainer;
    int capacity;
    int enrolled;
    float fee;
    char status;
    static int cnt;

public:
    void setData(int id, string name, string trainer,
                 int capacity, int enrolled, float fee)
    {
        this->id = id;
        this->name = name;
        this->trainer = trainer;
        this->capacity = capacity;
        this->enrolled = enrolled;
        this->fee = fee;

        if (enrolled == capacity)
            this->status = 'F';
        else
            this->status = 'O';
    }

    int getId() { return id; }

    inline int availableSeats()
    {
        return capacity - enrolled;
    }

    static int getCount() { return cnt; }
    static void increaseCount() { cnt++; }

    friend void show(GymClass &g);
};

int GymClass::cnt = 0;

void show(GymClass &g)
{
    cout << "\\nID: " << g.id;
    cout << "\\nClass Name: " << g.name;
    cout << "\\nTrainer: " << g.trainer;
    cout << "\\nCapacity: " << g.capacity;
```

11. SOURCE CODE – PART 2 OF 3

CA1 implementation using iostream, string and a fixed-size array. Records are maintained in memory during execution.

```
    cout << "\\nEnrolled: " << g.enrolled;
    cout << "\\nAvailable Seats: " << g.availableSeats();
    cout << "\\nFee: " << g.fee;
    cout << "\\nStatus: " << g.status << endl;
}

void addRecord(GymClass arr[], int &n)
{
    int id, capacity, enrolled;
    int flag = 0;
    float fee;
    string name, trainer;

    if (n >= 50)
    {
        cout << "\\nArray is full.\\n";
    }
    else
    {
        cout << "\\nEnter Class ID: ";
        while (!(cin >> id) || id <= 0)
        {
            cout << "Enter valid positive ID: ";
            if (cin.fail())
            {
                cin.clear();
                cin.ignore(1000, '\\n');
            }
        }

        for (int i = 0; i < n; i++)
        {
            if (arr[i].getId() == id)
            {
                flag = 1;
                break;
            }
        }

        if (flag == 1)
        {
            cout << "\\nDuplicate ID! Record not added.\\n";
        }
        else
        {
            cin.ignore();
            cout << "Enter Class Name: ";
            getline(cin, name);
            while (name.empty())
            {
                cout << "Enter class name: ";
                getline(cin, name);
            }
```

11. SOURCE CODE – PART 3 OF 3

CA1 implementation using iostream, string and a fixed-size array. Records are maintained in memory during execution.

```
            cout << "Enter Trainer Name: ";
            getline(cin, trainer);
            while (trainer.empty())
            {
                cout << "Enter trainer name: ";
                getline(cin, trainer);
            }

            cout << "Enter Maximum Capacity: ";
            while (!(cin >> capacity) || capacity <= 0)
            {
                cout << "Enter valid capacity: ";
                if (cin.fail())
                {
                    cin.clear();
                    cin.ignore(1000, '\\n');
                }
            }

            cout << "Enter Enrolled Members: ";
            while (!(cin >> enrolled) ||
                   enrolled < 0 || enrolled > capacity)
            {
                cout << "Enter valid enrolled members: ";
                if (cin.fail())
                {
                    cin.clear();
                    cin.ignore(1000, '\\n');
                }
            }

            cout << "Enter Class Fee: ";
            while (!(cin >> fee) || fee < 0)
            {
                cout << "Enter valid fee: ";
                if (cin.fail())
                {
                    cin.clear();
                    cin.ignore(1000, '\\n');
                }
            }

            arr[n].setData(id, name, trainer,
                           capacity, enrolled, fee);
            n++;
            GymClass::increaseCount();
            cout << "\\nRecord added successfully!\\n";
        }
    }
}

void displayAll(GymClass arr[], int n)
{
    if (n == 0)
    {
        cout << "\\nNo records found.\\n";
    }
    else
    {
        GymClass *ptr = arr;
        for (int i = 0; i < n; i++)
        {
            cout << "\\nRecord " << i + 1;
            show(*ptr);
            ptr++;
        }
        cout << "\\nTotal Records: "
             << GymClass::getCount() << endl;
    }
}

void searchRecord(GymClass arr[], int n)
{
    int key;
    int found = 0;
    cout << "\\nEnter ID to search: ";
```

```
        cout << "Enter valid ID: ";
        if (cin.fail())
        {
            cin.clear();
            cin.ignore(1000, '\\n');
        }
    }

    for (int i = 0; i < n; i++)
    {
        if (arr[i].getId() == key)
        {
            cout << "\\nRecord Found!\\n";
            show(arr[i]);
            found = 1;
            break;
        }
    }

    if (found == 0)
        cout << "\\nRecord not found.\\n";
}

int main()
{
    GymClass arr[50];
    int n = 0;
    int choice;

    do
    {
        cout << "\\n===== GYM CLASS MANAGER =====";
        cout << "\\n1. Add Gym Class";
        cout << "\\n2. Display All Classes";
        cout << "\\n3. Search Class by ID";
        cout << "\\n4. Exit";
        cout << "\\nEnter your choice: ";

        while (!(cin >> choice))
        {
            cout << "Enter valid choice: ";
            cin.clear();
            cin.ignore(1000, '\\n');
        }

        switch (choice)
        {
            case 1:
                addRecord(arr, n);
                break;
            case 2:
                displayAll(arr, n);
                break;
            case 3:
                searchRecord(arr, n);
                break;
            case 4:
                cout << "\\nThank you!\\n";
                break;
            default:
                cout << "\\nInvalid choice.\\n";
        }
    } while (choice != 4);

    return 0;
}
```

12. TEST CASES AND LIMITATIONS

12.1 Test Cases

Test Case	Input / Action	Expected Result
Add a class	New positive ID and valid details	Record added successfully
Duplicate ID	Enter an existing ID	Duplicate ID message; not added
Display records	Select option 2	All records displayed
Search existing ID	Enter ID present in array	Record Found and details displayed
Search unknown ID	Enter absent ID 999	Record not found
Full class	Enrolled equals capacity	Status is F (Full)
Available seats	Capacity 20, enrolled 15	5 available seats
Invalid capacity	Enter zero or negative	Program asks for valid capacity

12.2 Current Limitations

- Records are stored only in memory and are lost when the program closes.
- The fixed-size array can hold a maximum of 50 class records.
- The interface is text-based and menu-driven.
- Actual member registration and a waiting-list queue are not implemented in CA1.
- Update and delete operations are not included in this phase.

12.3 Future Scope

- Use file handling to save records permanently in CA2.
- Add constructors, destructors and binary file operations.
- Use inheritance, operator overloading and virtual functions in CA3.
- Add member registration and a FIFO waitlist in a later phase.
- Add update/delete operations, exception handling, STL containers and CSV backup in CA4.

13. CA1 PROGRESS SUMMARY & CONCLUSION

13.1 CA1 Progress Summary

CA1 is the Design & Create phase and is scheduled for seven days from the project allocation date. It covers Units I and II and focuses on the first working version of the application.

Task	Progress Status
Project topic confirmation	Completed
Problem identification	Completed
Entity and main class design	Completed
Data dictionary preparation	Completed
UML class diagram and flowchart	Prepared
CA1 source code	Prepared for compilation and testing
Test cases	Prepared; execute and attach actual evidence
Live demonstration and viva	To be completed

Update these statuses to match the work actually completed before submitting the report. Add screenshots of your own successful compilation and program execution as evidence.

13.2 Conclusion

The Gym Class Capacity and Waitlist Manager is a C++ project intended to organize gym class information and monitor class capacity. The CA1 phase establishes the basic application structure and demonstrates core Object-Oriented Programming concepts, including classes, objects, static members, inline functions, friend functions, arrays of objects, pointers, strings and the this pointer.

The current version supports adding, displaying and searching gym class records and checks duplicate IDs. It also calculates available seats and identifies whether a class is open or full. Records remain in memory during execution. Permanent storage and full waitlist operations are planned for later phases.

This phase provides a foundation for expanding the application into a more complete gym management system.

— END OF CA1 PROGRESS REPORT —